

Code Walkthrough

Reinforcement Learning Trading Environment — BTC/USDT

1. Overview

This project trains a PPO-based reinforcement learning agent to trade BTC/USDT on hourly data from August 2017 to April 2026. The agent operates in a semi-Markov environment where it decides whether to open a long, short, or stay flat at each decision point. Positions are managed automatically by the environment using stop-loss, take-profit, and maximum holding period rules. Walk-forward validation across four expanding folds is used to evaluate generalization.

2. Code Structure

The entire pipeline is in the `code/` directory and is run with a single entrypoint:

```
python -m code.main
```

File	Responsibility
<code>eda.py</code>	Loads raw CSV, computes regime analysis, return distributions, autocorrelation, and stationarity tests. Outputs <code>artifacts/btcusdt_eda.pdf</code> .
<code>features.py</code>	Computes all technical indicators and returns. Defines the four walk-forward fold splits and the normalization routine.
<code>environment.py</code>	Implements the gymnasium-compatible semi-Markov trading environment. Contains the position simulation logic with OHLC-based stop/take-profit resolution.
<code>train.py</code>	Runs the hyperparameter search on the validation set and retrains the best configuration for the full timestep budget. Saves models and training curves.
<code>benchmarks.py</code>	Computes BTC buy-and-hold and SPY equity curves for the test periods.
<code>evaluate.py</code>	Loads saved models, runs deterministic evaluation on test sets, computes all required metrics, and generates equity curves and trade logs.
<code>main.py</code>	Single entrypoint. Sets global random seeds and calls <code>run_training()</code> then <code>run_evaluation()</code> .
<code>generate_walkthrough.py</code>	Generates this document.

3. Environment Design

3.1 Semi-Markov Formulation

The environment is semi-Markov: the agent only makes a decision when it has no open position. Once a position is opened, the environment simulates forward bar by bar until a closing condition is met. Each gymnasium step therefore covers a variable number of hourly bars. PPO handles this naturally since it treats each step as a single transition regardless of the real-world duration.

3.2 Action Space

Discrete(3): action 0 = stay flat (advance one bar), action 1 = open long, action 2 = open short. A continuous action space (e.g., position size) was considered but rejected because it would require a more complex reward normalization and the project focuses on direction prediction rather than sizing.

3.3 Entry and Exit Price Assumptions

Entry price: the open of the bar immediately following the decision bar. This avoids any within-bar lookahead on the decision bar. Exit price: exactly at the stop-loss or take-profit level if triggered within a bar, or at the close of the bar if the horizon is reached. OHLC assumption: if both stop-loss and take-profit are touched within the same bar, the stop-loss fills first. This is the conservative default and is explicitly stated in the environment docstring.

3.4 Episode Definition

Each training episode is a 720-bar window (30 days) with a randomly sampled start index within the training fold. This gives the agent diverse market conditions across episodes without memorizing a fixed sequence. A fixed-start single-episode approach was considered but would have led to severe overfitting to the specific trajectory seen during training.

4. Feature Engineering

All 17 features are causally computable at decision time — no future data is used. Features fall into five categories:

Category	Features	Justification
Returns	log_return, log_return_2h/4h/8h/24h	Captures recent momentum. Autocorrelation in the EDA showed returns are near-uncorrelated at lag 1, but multi-period returns carry some signal (MI > 0.05).
Volatility	vol_24h, vol_72h, vol_168h	Justified by strong ACF in squared returns (volatility clustering). ATR-based feature also included for position sizing context.
Momentum	rsi_14, rsi_28, macd_diff	Standard momentum indicators. Low MI scores suggest limited predictive power but included for completeness.
Structure	bb_pct, bb_width, atr_pct	Bollinger band position and width capture mean-reversion and volatility expansion regimes.
Time	hour_sin, hour_cos	Crypto trades 24/7. Cyclical encoding preserves continuity at hour 23 to 0. Low MI suggests intraday patterns are weak in this dataset.

Normalization: means and standard deviations are computed on the training fold only and applied to validation and test folds. This prevents any information leakage from future data into the feature scaling.

5. Training Procedure

Algorithm: PPO (Proximal Policy Optimization) with MlpPolicy. PPO was chosen over SAC because our action space is discrete — SAC is designed for continuous actions. DDPG was rejected for the same reason. A2C was considered but PPO's clipped objective consistently outperforms A2C in practice.

Hyperparameter search: three configurations are evaluated on the validation fold for 50,000 steps each. The configuration with the highest cumulative reward on the validation set is retrained for 500,000 steps. The search grid varies learning rate, entropy coefficient, stop-loss, and take-profit simultaneously.

Fixed hyperparameters across all configurations: batch_size=64, n_epochs=10, gamma=0.99, gae_lambda=0.95, clip_range=0.2, n_steps=2048. These are the SB3 defaults and were not tuned to keep the search space manageable.

6. Walk-Forward Evaluation

Four expanding-window folds are used. Training always starts from August 2017. The validation set is used only for hyperparameter selection. The test set is touched exactly once after all hyperparameter decisions are finalized. Models are saved per fold and evaluation is run separately from training to allow re-evaluation without retraining.

Fold	Train End	Validation	Test	Train Rows	Test Rows
1	Dec 2021	2022	2023	38,054	8,759
2	Dec 2022	2023	2024	46,814	8,784
3	Dec 2023	2024	2025	55,573	8,579
4	Dec 2024	2025	Jan-Apr 2026	64,357	2,484

7. Reproducibility

All random seeds are fixed at 42 via set_seeds() in main.py, which sets Python random, NumPy, and PyTorch seeds. SB3 receives seed=42 directly. Running python -m code.main from the project root with the dataset at data/Binance_BTCUSDT_1h.csv reproduces all artifacts in a single pass.